AI Depression Analysis System

Deployment Presentation

Agile Production Development

FastAPI | GitHub CI/CD | Docker | Azure

Dhruv Marwal(I043) | Priyanshu Jha (I034) | Shivang Jain (I030)

Project Overview

What we built and why we deployed it

The Project

- AI-powered web application for automated depression screening
- Users answer PHQ-9 questionnaire + chat interface
- Backend: FastAPI (Python) with ML models
- Frontend: HTML + JavaScript UI
- Goal: Deploy as scalable, public-facing cloud application

Deployment Goals

- Make the app accessible from anywhere via public URL
- Containerize for environment consistency
- Automate future deployments with CI/CD

Tech Stack

Frontend
HTML / JavaScript

API Layer
FastAPI (Python)

ML Models
PyTorch · Transformers · DEAP

Container
Docker

Registry
Azure Container Registry (ACR)

Cloud Hosting
Azure App Service (B1 Linux)

CI/CD
GitHub Actions

System Architecture

End-to-end architecture of the deployed system

User Browser
HTML/JS

FastAPI Backend
Python 3.10

ML Modules
PHQ9·Sentiment·Fuzzy

Docker Container
neurofuzzy-app

Azure ACR
Image Registry

Azure App Service
Public URL

Backend Modules

- PHQ-9 Scorer – questionnaire analysis
- Sentiment Analyzer – emotion from chat
- Fuzzy Logic Engine – inference system
- Classifier – final depression category

Deployment Infrastructure

- Docker image: neurofuzzy-app:latest
- ACR: neuroregistry123.azurecr.io
- App Service Plan: B1 (Linux)
- Port: 8000 → WEBSITES_PORT

CI/CD Pipeline

- GitHub Actions on push to main
- Auto docker build + push to ACR
- Auto Azure Web App restart
- Zero-touch deployment

Backend Development (FastAPI)

Building the core API server in Python

Why FastAPI?

- Modern, high-performance Python web framework
- Automatic Swagger UI at /docs
- Built-in async support
- Easy integration with ML models
- JSON API responses out of the box

```python
from fastapi import FastAPI, HTTPException, UploadFile, File
from fastapi.middleware.cors import CORSMiddleware
from fastapi.responses import JSONResponse

# ── App Setup ──────────────────────────────────────────────────────────
app = FastAPI(
    title       = "Depression Analysis API",
    description= "PHQ-9 + Sentiment + GA/Fuzzy Logic Depression Assessment",
    version     = "1.0.0",
)

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],          # In production, restrict to your frontend domain
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

CORS Configuration

- CORSMiddleware added to allow all origins
- Required for frontend-backend cross-origin calls
- allow_origins=["*"] for development
- Prevents 'Failed to fetch' errors

App Structure

- Project root → backend/ + frontend/ + Dockerfile + requirements.txt
- Uvicorn ASGI server: uvicorn backend.api:app --host 0.0.0.0 --port 8000
- ML models loaded at startup (PHQ9 scorer, sentiment, classifier)
- Frontend served from FastAPI static file response at root route /

API Endpoints Design

RESTful endpoints powering the depression analysis system

Depression Analysis API 1.0.0 OAS 3.1

/openapi.json

PHQ-9 + Sentiment + GA/Fuzzy Logic Depression Assessment

default

GET	/ Serve Frontend	
GET	/health Health Check	Serves the frontend index.html file
POST	/analyze/phq9 Analyze Phq9	Returns server status and loaded modules
POST	/analyze/chat Analyze Chat	Full depression analysis (PHQ-9 + chat)
POST	/analyze/full Full Analysis	Sentiment analysis from conversation text
POST	/speech/transcribe Transcribe Audio	Standalone PHQ-9 questionnaire scoring

Local Testing & Validation

Verifying the app works before containerizing

Why Test Locally First?

- Catch errors before adding Docker complexity
- Faster iteration without image rebuild cycles
- Verify ML models load and run correctly
- Test all API endpoints with Swagger UI at /docs
- Ensure frontend connects to backend successfully

Validation Steps

- Run: uvicorn backend.api:app --port 8000
- Open: http://localhost:8000/docs
- Test /health → modules loaded?
- Test /analyze endpoints with sample data
- Confirm frontend UI loads and calls API

$ terminal

```
# Start server locally
uvicorn backend.api:app --host 0.0.0.0 --port 8000

# Expected output:
INFO: Application startup complete.
INFO: Uvicorn running on http://0.0.0.0:8000

# Test endpoints:
curl http://localhost:8000/health
# → {"status":"ok","modules":["phq9_scorer",...]}

curl -X POST http://localhost:8000/analyze/chat \
  -d '{"text":"I feel hopeless today"}'
```

Local validation passed:

Swagger UI responsive • Models load in ~30s • All endpoints return JSON

Introduction to Docker

Containerization — packaging the app for any environment

What is Docker?

- Platform to build, ship and run applications in containers
- A container packages code + dependencies + OS layer
- Runs identically on any machine: dev, staging, cloud
- Eliminates 'works on my machine' problem

Why Docker for our project?

- ML models require exact Python + library versions
- Azure App Service natively supports Docker containers
- Single image can be versioned (v1, v2 ... latest)
- Easy rollback to previous version if deploy fails

Container vs Traditional Deploy

Aspect	Traditional	Docker
Setup	Manual install	One command
Environment	Varies per machine	Always consistent
Deploy	Copy files + hope	docker run
Scale	Complex	Simple

Docker Key Concepts Used

- Image – blueprint of our app (built with Dockerfile)
- Container – running instance of the image
- Tag – version label (v1, v2, latest)
- Registry – remote image storage (Azure ACR)

Dockerfile Setup

Defining how our app container is built

```
$ terminal
```

```
# Use lightweight Python base image
FROM python:3.10-slim

# Set working directory inside container
WORKDIR /app

# Copy dependency list first (layer cache optimization)
COPY requirements.txt .

# Install Python dependencies
RUN pip install --no-cache-dir -r requirements.txt

# Install CPU-only PyTorch (smaller image size)
RUN pip install --no-cache-dir \
  --extra-index-url https://download.pytorch.org/whl/cpu \
  -r requirements.txt

# Copy entire project (backend/ + frontend/)
COPY . .

# Expose app port
EXPOSE 8000

# Start FastAPI with Uvicorn
CMD ["uvicorn", "backend.api:app",
     "--host", "0.0.0.0", "--port", "8000"]
```

Base Image Choice

- python:3.10-slim → minimal, no GUI
- Reduces image size significantly
- All ML libs installable via pip
- Compatible with Azure Linux containers

Layer Caching Strategy

- COPY requirements.txt FIRST
- RUN pip install as a separate layer
- Docker caches if requirements unchanged
- Subsequent builds skip reinstall → faster

CPU-only PyTorch

- torch==2.3.0+cpu → removes CUDA packages
- Reduces image from ~5GB to ~1.85GB
- Azure App Service has no GPU access
- Faster container startup in cloud

Building Docker Image

Creating the container image from our project

$ terminal

```
# Navigate to project root
cd C:\Codes\neurofuzzy-depression-analysis

# Build Docker image (tag it "neurofuzzy-app")
docker build -t neurofuzzy-app .

# Verify image was created
docker images
# REPOSITORY       TAG    IMAGE ID     SIZE
# neurofuzzy-app   latest 24d1218c...  1.85GB
```

$ terminal

```
# Run container locally to test before cloud deploy
docker run -p 8000:8000 neurofuzzy-app

# Expected console output:
INFO: Application startup complete.
INFO: Uvicorn running on http://0.0.0.0:8000

# Test in browser:
# http://localhost:8000/docs → Swagger UI
```

Build Command Explained

- docker build → triggers Dockerfile
- -t neurofuzzy-app → assigns name tag
- . (dot) → build context = current folder
- Copies all project files including frontend/
- Executes RUN commands, sets CMD

Run Command Explained

- docker run → start container from image
- -p 8000:8000 → host:container port mapping
- Maps laptop port 8000 → container port 8000
- If local test passes → Azure will work too
- Confirms image is complete and functional

Key Insight: If the app runs locally in Docker → it will run identically on Azure. This is the core promise of containerization.

Handling Dependency Issues

Real problems encountered and how we solved them

Issue 1: pyaudio Build Failure

Problem: error: command 'gcc' failed — pyaudio requires C compiler to build

Fix: Removed pyaudio + SpeechRecognition from requirements.txt
Cloud has no microphone hardware — audio input not needed on Azure

Issue 2: Huge Docker Image (~5GB)

Problem: torch default install includes NVIDIA CUDA (~2GB extra). Azure B1 plan crashes on memory overload.

Fix: Used CPU-only PyTorch:
--extra-index-url https://download.pytorch.org/whl/cpu
torch==2.3.0+cpu → Image shrunk to 1.85GB

Issue 3: requirements.txt Syntax Error

Problem: Wrote torch==2.3.0+cpu --extra-index-url on SAME line → pip parse error

Fix: Moved --extra-index-url to Dockerfile RUN command separately:
RUN pip install --extra-index-url https://...whl/cpu -r requirements.txt

Docker Image Versioning (Tags)

Managing multiple versions of our deployment

What are Tags?

- Labels attached to Docker images for version tracking
- Format: repository:tag (e.g. neurofuzzy-app:v3)
- Default tag is 'latest' if none specified
- Multiple tags can point to same image ID
- Critical for rollback and reproducibility

$ terminal

```
# Tagging images for Azure ACR
docker tag neurofuzzy-app \
  neuroregistry123.azurecr.io/neurofuzzy-app:v1

docker tag neurofuzzy-app \
  neuroregistry123.azurecr.io/neurofuzzy-app:v3

# For CI/CD automation (always latest)
docker tag neurofuzzy-app \
  neuroregistry123.azurecr.io/neurofuzzy-app:latest

# Our version progression:
# v1 → initial deploy
# v2 → Dockerfile CMD fix
# v3 → CMD order fix
# v5–v8 → frontend integration fixes
# latest → CI/CD automated builds
```

Version Timeline

v1	Initial push to ACR
v2	Fixed Dockerfile CMD order
v3	Removed duplicate CMD statements
v5	Fixed frontend folder not included
v7	Added safe FileResponse with path check
v8	Fixed API BASE_URL for production
latest	CI/CD auto-build on every git push

Pushing Image to Azure Container Registry (ACR)

Uploading our Docker image to the cloud registry

What is ACR?

- Azure's private Docker image registry
- Like Docker Hub but private to our Azure account
- Azure App Service pulls images directly from ACR
- Supports access credentials and admin login

ACR Admin Credentials

- 2 passwords generated (primary + secondary)
- Password 1: primary usage
- Password 2: backup for rotation (zero-downtime key change)
- Used by Azure Web App to pull images securely

Push Success Confirmation

- digest: sha256:... confirms image integrity
- Layer already exists = efficient incremental push
- Only changed layers are re-uploaded
- Registry URL: neuroregistry123.azurecr.io

$ terminal

```
# STEP 1: Create Container Registry
az acr create --resource-group neuro-rg \
  --name neuroregistry123 --sku Basic

# STEP 2: Enable admin credentials
az acr update -n neuroregistry123 --admin-enabled true

# STEP 3: Login to ACR from local machine
az acr login --name neuroregistry123

# STEP 4: Tag image with full ACR path
docker tag neurofuzzy-app \
  neuroregistry123.azurecr.io/neurofuzzy-app:v1

# STEP 5: Push image to ACR
docker push neuroregistry123.azurecr.io/neurofuzzy-app:v1

# Expected output:
# v1: digest: sha256:cdb3bf33... size: 856
#    Push complete
```

Azure Cloud Setup (Resource Group & App Service)

Setting up the cloud infrastructure for deployment

```
$ terminal
# STEP 1: Login to Azure CLI
az login --use-device-code
# Visit https://microsoft.com/devicelogin
# Enter code → authenticate

# STEP 2: Create Resource Group
az group create --name neuro-rg \
  --location centralindia
# Groups all project resources together

# STEP 3: Create App Service Plan (Linux B1)
az appservice plan create \
  --name neuro-plan \
  --resource-group neuro-rg \
  --sku B1 --is-linux

# STEP 4: Create Web App (container-based)
az webapp create \
  --resource-group neuro-rg \
  --plan neuro-plan \
  --name neurofuzzy-depression \
  --deployment-container-image-name \
  neuroregistry123.azurecr.io/neurofuzzy-app:v1
```

Resource Group

- Logical container for all Azure resources
- Name: neuro-rg | Region: Central India
- Contains: ACR + App Service Plan + Web App
- Delete group → deletes all resources at once

App Service Plan (B1)

- Defines compute power: 1 core, 1.75GB RAM
- B1 = Basic tier — supports custom containers
- --is-linux required for Docker containers
- Shared by all Web Apps under the plan

Azure CLI Note

- Use single-line commands in PowerShell
- Line continuation: backtick (`) not backslash
- az --version to verify CLI installation
- az account show to confirm subscription

Deploying Container to Azure Web App

Connecting the Docker image to Azure App Service

Deployment Center Setup (Azure Portal)

- App Service → Deployment Center → Containers
- Source: Azure Container Registry
- Registry: neuroregistry123
- Image: neurofuzzy-app
- Tag: v8 (or latest for CI/CD)
- Auth: Admin Credentials
- Click Save → Azure pulls image automatically

```
$ terminal
# Configure container source via CLI
az webapp config container set \
  --name neurofuzzy-depression \
  --resource-group neuro-rg \
  --container-image-name \
    neuroregistry123.azurecr.io/neurofuzzy-app:v1 \
  --container-registry-url \
    https://neuroregistry123.azurecr.io \
  --container-registry-user neuroregistry123 \
  --container-registry-password <PASSWORD>

# Restart to apply changes
az webapp restart \
  --name neurofuzzy-depression \
  --resource-group neuro-rg
```

Deployment Flow

- 1. Azure pulls image from ACR using credentials
- 2. Container is started on Linux B1 instance
- 3. Uvicorn serves FastAPI on port 8000
- 4. Azure proxies port 80/443 → port 8000
- 5. Public URL becomes accessible globally
- URL: https://neurofuzzy-depression.azurewebsites.net

Container Status States

- Starting → container boot in progress
- Running → app serving requests
- Unknown → timeout or crash during boot
- Blocked → SiteStartupCancelled (timeout hit)
- Stopped → app crashed after startup
- Fix: check Log Stream for exact error

Configuring Port & Environment Variables

Critical Azure settings for container to work correctly

Name	Value	Why It's Needed
DOCKER_CUSTOM_IMAGE_NAME	neuroregistry123.azurecr.io/neurofuzzy-app:v1	
DOCKER_REGISTRY_SERVER_PASSWORD	Show value	ACR admin password (sensitive credential)
DOCKER_REGISTRY_SERVER_URL	https://neuroregistry123.azurecr.io	ACR server URL for image pull authentication
DOCKER_REGISTRY_SERVER_USERNAME	neuroregistry123	ACR admin username for authentication
WEBSITES_CONTAINER_START_TIME_LIMIT	1800	Extends startup timeout to 30min for heavy ML model loading
WEBSITES_ENABLE_APP_SERVICE_STORAGE	false	
WEBSITES_PORT	8000	Tells Azure which port the container listens on (default is 80)

$ terminal

```
az webapp config appsettings set --name neurofuzzy-depression \
  --resource-group neuro-rg \
  --settings WEBSITES_PORT=8000 WEBSITES_CONTAINER_START_TIME_LIMIT=1800
```

Key Lesson

- Azure proxies port 80→8000 via WEBSITES_PORT
- Without this: 'Application Error' on every request
- Startup timeout fix essential for ML models

Debugging with Azure Log Stream

Reading real-time logs to diagnose deployment issues

What is Log Stream?

- Real-time live log viewer inside Azure Portal
- Shows container stdout/stderr in browser
- Path: App Service → Log stream (left sidebar)
- Also accessible via Azure CLI
- Essential for diagnosing container crashes

$ terminal

```
# View live logs from CLI
az webapp log tail \
  --name neurofuzzy-depression \
  --resource-group neuro-rg

# Common success messages:
INFO: Application startup complete.
INFO: Uvicorn running on http://0.0.0.0:8000

# Common failure messages:
ContainerTimeout → startup > 230s
'Killed' → Out of memory
ModuleNotFoundError → missing package
```

Log Messages We Encountered

`ContainerTimeout (230 sec expired)`

→ Increased start limit to 1800s

`Uvicorn running on http://0.0.0.0:8000`

→ App started successfully

`Application startup complete.`

→ All models loaded

`No credential was provided to access ACR`

→ Set DOCKER_REGISTRY_SERVER_PASSWORD

`GET / HTTP/1.1 404 Not Found`

→ Normal — no route defined at /

`Shutting down / Application shutdown complete`

→ Azure killing slow-start container

`{"status":"ok","modules":[...]}`

→ Health check passed

Frontend Integration with Backend API

Connecting the HTML/JS UI to our deployed API

$ terminal

```
//    WRONG — won't work in production
const API = "http://localhost:8000";
//    CORRECT — points to live Azure URL
const API = "https://neurofuzzy-depression.azurewebsites.net";
// API calls use the variable (no other changes needed)
fetch(`${API}/analyze/phq9`, {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify(phq9Data)
});
```

CORS — Why It Matters

- Browser blocks cross-origin API requests by default
- Frontend on domain A → calls API on domain B = blocked
- FastAPI CORSMiddleware allows our frontend to call API
- allow_origins=['*'] permits all origins for dev
- Without this: 'Failed to fetch' errors in browser console

Serving Frontend from FastAPI

- FastAPI serves index.html at GET / route
- os.path.join(os.getcwd(), 'frontend', 'index.html')
- FileResponse streams HTML file to browser
- Docker COPY . . includes entire frontend/ folder
- No separate web server (nginx) needed

Integration Architecture

- Browser loads index.html from Azure App URL
- JS form submission → fetch(API + /analyze/full)
- FastAPI processes → returns JSON result
- Frontend renders depression analysis result
- Same domain = no CORS issues in our setup

Pro Tip for future:

Use const API = window.location.origin; — automatically detects localhost OR production Azure URL, making the code work in both environments without changes.

Fixing API Connectivity Issues

Diagnosing and resolving frontend-backend connection problems

Error: "Failed to fetch"

Cause: Frontend JavaScript was calling http://localhost:8000 — which doesn't exist on Azure's servers

Fix: Changed: const API = "https://neurofuzzy-depression.azurewebsites.net"
Rebuild Docker image → push v8 → update Azure → restart

Error: 500 Internal Server Error on GET /

Cause: FileResponse('frontend/index.html') — relative path failed because working directory mismatch inside container

Fix: Used os.path.join(os.getcwd(), 'frontend', 'index.html') for absolute path
Added existence check to return helpful error instead of crash

404 on /frontend/index.html

Cause: Initially thought frontend folder wasn't in container. Confirmed it was included (COPY . . copies all), but path resolution was wrong

Fix: Fixed FileResponse to use absolute path. Static route at / now correctly returns index.html

Successful Deployment Verification

Confirming the live system works end-to-end

App Live at: https://neurofuzzy-depression.azurewebsites.net

Endpoint	Verification Result
`/health`	{"status":"ok","modules":["phq9_scorer","sentiment_analyzer","classifier"]}
`/docs`	Swagger UI loaded — all 5 API endpoints listed and testable
`/`	Frontend HTML page loads correctly in browser
`/analyze/full`	POST with PHQ-9 data returns depression classification JSON
`/analyze/chat`	POST with chat text returns sentiment + depression score

- Runtime Status: Healthy
- Container: v8 running
- API modules: 3 loaded
- First response time: ~30s (model loading)
- Subsequent: <2s

Challenges Faced During Deployment

Real-world obstacles encountered on the path to production

Docker Build

- pyaudio requires gcc compiler not present in slim image
- torch default pulls 2GB CUDA — container too heavy
- Docker Desktop not running: daemon connection error
- CMD specified twice in Dockerfile — last one silently wins

Container Startup

- ContainerTimeout after 230 seconds — ML models load too slowly
- Status: Unknown — Azure couldn't confirm app started
- Startup command location changed in new Azure UI
- WEBSITES_PORT not set — Azure tried wrong port (80 vs 8000)

Azure Setup

- az CLI not recognized (not installed on Windows)
- az login browser sync failed — needed --use-device-code
- ACR admin password was null — container couldn't pull image
- PowerShell: ^ multiline doesn't work, need backtick or single line

Frontend-Backend

- localhost:8000 hardcoded — failed in production
- Frontend folder not served (FileResponse path issue)
- CORS errors when calling API from browser
- Browser cache serving old JS with old localhost URL

Solutions & Fixes Implemented

How every challenge was diagnosed and resolved

pyaudio build fail

Removed pyaudio from requirements.txt — cloud has no microphone hardware

Oversized Docker image

Used torch==2.3.0+cpu with --extra-index-url → image from 5GB to 1.85GB

Container startup timeout

Set WEBSITES_CONTAINER_START_TIME_LIMIT=1800 (30 minutes for ML models)

ACR auth null password

Used az acr credential show → explicitly set DOCKER_REGISTRY_SERVER_PASSWORD

Wrong frontend URL

Set const API = "https://neurofuzzy-depression.azurewebsites.net" in JS

FileResponse internal error

Used os.path.join(os.getcwd(), 'frontend', 'index.html') for absolute path

WEBSITES_PORT missing

Added WEBSITES_PORT=8000 in Configuration → Application Settings

Introduction to CI/CD Pipeline

Automating the build-test-deploy cycle with GitHub Actions

What is CI/CD?

- CI = Continuous Integration: auto-build on code push
- CD = Continuous Deployment: auto-release to production
- Goal: zero-touch deployment on every git push
- Eliminates manual docker build → tag → push → restart cycle
- Industry standard practice in all production systems

Why GitHub Actions?

- Native integration with GitHub repositories
- Free for public repos, minutes-based for private
- YAML-defined workflow — version controlled with code
- Rich marketplace of reusable actions (azure/login, etc.)
- Easiest setup for GitHub → Azure pipeline

Before CI/CD (Manual)

1. Edit code locally
2. docker build -t neurofuzzy-app .
3. docker tag ... :v9
4. docker push ...
5. Azure portal → change tag
6. Save → Restart
7. Wait 2 min → check /health

~15-20 minutes, error-prone

After CI/CD (Automated)

1. Edit code locally
2. git add . && git commit -m 'fix'
3. git push

~3 minutes, fully automatic

CI/CD Tools Comparison

- GitHub Actions → easiest setup, YAML-based, GitHub-native
- Jenkins → enterprise, self-hosted, more config needed
- Azure DevOps → Microsoft ecosystem, powerful pipelines

Automating Deployment using GitHub Actions

The YAML workflow that powers our CI/CD pipeline

$ terminal

```yaml
# .github/workflows/deploy.yml
name: Deploy to Azure

on:
  push:
    branches:
      - main       # Triggers on every push to main

jobs:
  build-and-deploy:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout code
        uses: actions/checkout@v3

      - name: Login to Azure
        uses: azure/login@v1
        with:
          creds: ${{ secrets.AZURE_CREDENTIALS }}

      - name: Login to ACR
        run: az acr login --name neuroregistry123

      - name: Build Docker image
        run: |
          docker build -t \
          neuroregistry123.azurecr.io/neurofuzzy-app:latest .

      - name: Push Docker image
        run: |
          docker push \
          neuroregistry123.azurecr.io/neurofuzzy-app:latest

      - name: Restart Azure Web App
        run: |
          az webapp restart \
            --name neurofuzzy-depression \
            --resource-group neuro-rg
```

GitHub Repository Secrets

- Settings → Secrets → Actions → New repository secret
- AZURE_CREDENTIALS = full JSON from:
- az ad sp create-for-rbac --name 'github-action'
- --role contributor --scopes /subscriptions/ID --sdk-auth

Workflow Triggers

- on: push: branches: [main] — runs on main branch push
- Can also trigger on: pull_request, schedule, manual
- Each push creates a new Actions run visible in UI
- Logs available in GitHub → Actions tab

Setup Steps Summary

- 1. Create .github/workflows/deploy.yml
- 2. Add AZURE_CREDENTIALS GitHub secret
- 3. Enable Continuous deployment in Azure portal
- 4. First push triggers full pipeline

CI/CD Workflow Overview

Visual step-by-step pipeline breakdown

1
Developer Pushes Code
git push
to main branch

2
GitHub Actions Triggered
on: push
workflow starts

3
Docker Image Built
docker build
on ubuntu runner

4
Image Pushed to ACR
docker push
:latest tag

5
Azure App Restarted
az webapp restart
pulls new image

What GitHub Actions Runs (Automated):

```
$ terminal
actions/checkout@v3        →   Clone repo to runner
azure/login@v1             →   Authenticate to Azure using AZURE_CREDENTIALS secret
az acr login               →   Authenticate Docker to Azure Container Registry
docker build               →   Build new image with latest code changes
docker push :latest        →   Upload image to ACR (only changed layers)
az webapp restart          →   Force Azure to pull new :latest image and restart
```

Total Pipeline Time

- ~3–5 minutes end-to-end (build + push + deploy)
- Visible in GitHub → Actions tab with live logs

Azure Continuous Deployment

- Enable in Deployment Center → triggers on ACR push webhook
- No az webapp restart needed if webhook is active

End-to-End Deployment Flow (Git → Azure)

Complete journey from code change to live production

1	2	3	4	5	6
Local Dev	**GitHub**	**GitHub Actions Runner**	**Azure ACR**	**Azure App Service**	**Users**
Edit code git add . && git commit	git push → repo updated Workflow triggered by on:push	Checkout → Docker build → Push to ACR	Stores :latest image Webhook notifies App Service	Pulls new image → restart New container running	Access updated app at neurofuzzy.azurewebsites.net

Full Command Sequence (One-Time Setup Done → Now Just 3 Commands)

- git add . && git commit -m 'your message' && git push
- GitHub Actions runs automatically → Docker builds → ACR push → Azure restarts → App updated in ~3–5 min

Our Deployment Journey — Version History

v1	v2-v3	v5-v6	v7-v8	latest
Initial Deploy	Dockerfile Fixes	Frontend Integration	API URL Fix	CI/CD Automatic

Key Learnings

Technical insights gained from the full deployment experience

Docker

- Layer caching: copy requirements.txt before full project for faster builds
- CPU-only ML dependencies dramatically reduce image size
- CMD must be the LAST instruction — duplicates silently ignored
- .dockerignore prevents unnecessary files from entering the image

Debugging

- Absolute paths > relative paths inside containers
- Always test locally in Docker before pushing to cloud
- One environment variable typo can cascade into many failures
- Browser cache masks recent deployments — always hard refresh

Azure

- WEBSITES_PORT is mandatory — Azure doesn't auto-detect container port
- Container startup timeout must be extended for heavy ML models
- Admin credentials required for ACR pull — managed identity is cleaner
- Log Stream is the fastest debugging tool for container issues

CI/CD

- GitHub Actions secrets securely store Azure credentials
- latest tag enables automatic deployment without version management
- Webhook-based continuous deployment removes manual restart step
- Pipeline failures are visible and debuggable in GitHub Actions tab

Future Improvements

Enhancements to make the deployment more robust and scalable

Jenkins Pipeline

Replace GitHub Actions with Jenkins for enterprise-grade CI/CD with self-hosted runner, build history, and plugin ecosystem.

Azure Managed Identity

Replace admin password credentials with Azure Managed Identity for ACR access — more secure, no secrets to manage.

Separate Frontend on Vercel/Netlify

Host HTML/JS frontend on Vercel (free tier) — zero cost, global CDN, instant deploy. Backend stays on Azure.

Custom Domain + SSL

Purchase custom domain (e.g. mindsense.ai), configure CNAME in Azure, enable HTTPS with auto-certificate renewal.

Lazy Model Loading

Load ML models on first API request rather than startup. Reduces container boot time from 60s to <5s, prevents Azure timeouts.

Monitoring & Alerts

Integrate Azure Application Insights for request tracking, error alerts, and performance monitoring with real-time dashboards.

Conclusion

Summary of the complete deployment journey

What We Accomplished

FastAPI Backend

Built a production-ready Python API with ML models, CORS, and static file serving

Docker Containerization

Packaged the entire app (backend + frontend + dependencies) into a portable 1.85GB image

Azure ACR + App Service

Pushed image to cloud registry and deployed as a globally accessible web app

End-to-End Debugging

Diagnosed and resolved 10+ real-world deployment issues from timeouts to CORS to path errors

CI/CD with GitHub Actions

Automated the full pipeline: code change → git push → live update in 3–5 minutes

This project demonstrates a complete Agile Production Deployment lifecycle using Docker, Azure, and GitHub Actions CI/CD — exactly the industry-standard DevOps pipeline.

Thank You

Questions & Discussion

Docker | Azure ACR | App Service | GitHub Actions | FastAPI | Python

Live URL: https://neurofuzzy-depression.azurewebsites.net

GitHub: github.com/dhruvmarwal/neurofuzzy-depression-analysis